

Sebastian Vintoñuke – 106063

Cindy Teresa Hsieh – 108051

Maria Delfina Cano Ros Langrehr – 109338

Lautaro Martin Sotelo – 107472

Toda afirmación que se realice debe estar apoyada en **evidencia**. Si se encuentra alguna **afirmación** que no está correctamente fundamentada con **evidencia**, se disminuirá la nota. Esto incluye las observaciones que realicen sobre el código y su funcionamiento en general. Que no existan métricas para un determinado QA no implica que no pueda evidenciarse el efecto de una táctica.

1. SISTEMA RECIBIDO

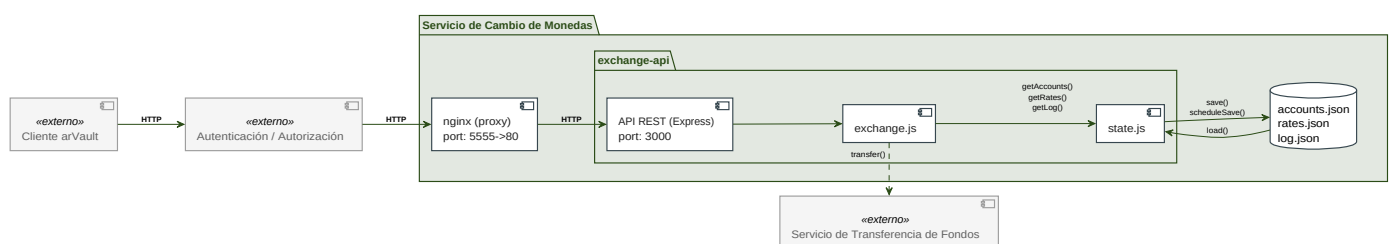


Figura 1. Diagrama Components & Connectors del sistema base.

1.1. Arquitectura

Arquitectura monolítica cliente-servidor, formada por una única instancia app.js. Separación en tres capas: app.js punto de entrada y definición de endpoints, exchange.js lógica de negocio, state.js persistencia. No se implementa coordinación, no hay locks ni mecanismos de concurrencia sobre secciones críticas, como la verificación del saldo. No se implementa tolerancia a fallos, no hay manejo de errores try/catch. No hay replicación, única instancia, no hay réplicas. No hay recovery, si el servidor muere hay que levantarlo manualmente. No hay consistencia entre los datos en memoria y los datos en disco, la consistencia es eventual sin mecanismos de transaccionabilidad.

TODO: redactarlo mas bonito y justificar

1.2. Código

No hay separación de responsabilidades, exchange maneja tanto lógica de negocio, como lógica de bajo nivel. Calcula tanto counterAmount como se encarga de persistir los cambios en counterAccount.balance.

```
1 export async function exchange(exchangeRequest) {
2   ...
3   //get the exchange rate
4   const exchangeRate = rates[baseCurrency][counterCurrency];
5   //compute the requested (counter) amount
6   const counterAmount = baseAmount * exchangeRate;
7   ...
8   //all good, update balances
9   baseAccount.balance += baseAmount;
10  counterAccount.balance -= counterAmount;
11  ...
12 }
```

Código 1. app/exchange.js

Hay alto acoplamiento, exchange.js accede directamente a variables que deberían ser privadas de state.js. exchange.js no debería conocer detalles de implementación de la persistencia, se debería programar contra interfaces. Cambiar la persistencia implica modificar exchange.js. No hay modelo de clases, todo está basado en constantes globales y funciones públicas.

```
1 let accounts;
2 let rates;
3 let log;
4
5 export async function init() {
6   await stateInit();
7 }
```

```

8   accounts = stateAccounts();
9   rates = stateRates();
10  log = stateLog();
11 }

```

Código 2. app/exchange.js

No se implementa transaccionalidad, posibles inconsistencias ante fallos incluso al recuperarse.

```

1  async function save(data, fileName) {
2    const filePath = path.join(__dirname, fileName);
3    try {
4      await fs.promises.writeFile(filePath, JSON.stringify(data, null, 2));
5    } catch (err) {
6      console.error(`Error writing to ${filePath}:`, err);
7    }
8  }

```

Código 3. app/state.js

No se implemento un diseño guiado por dominio, se opera directamente contra los tipos de datos primitivos. Se modela el dinero utilizando floats, esto genera perdida de precision.

```

1  const exchangeRate = rates[baseCurrency][counterCurrency];
2  //compute the requested (counter) amount
3  const counterAmount = baseAmount * exchangeRate;

```

Código 4. app/exchange.js

1.3. Infraestructura

2. ATRIBUTOS DE CALIDAD

2.1. Identificación de los QA claves

Se identifican los siguientes atributos de calidad como claves en el contexto del negocio. Adicionalmente, se citan especificaciones y requerimientos que motivaron estas afirmaciones.

2.1.1. Availability (disponibilidad)

Habilidad del sistema para reparar o enmascarar fallas de manera tal que el período fuera de servicio no exceda un determinado valor durante un lapso de tiempo especificado.

La startup arVault es una fintech que opera una billetera digital

Al ser una billetera digital, el servicio permite manipular el dinero de los usuarios. Un servicio con baja o nula disponibilidad implicaría que los usuarios no pueden acceder a su propio dinero. Adicionalmente, el modelo de negocio se basa en que los usuarios operen, si el servicio no es disponible para permitir estas operaciones, se estan perdiendo potenciales ganancias.

2.1.2. Performance

Habilidad del sistema para reaccionar ante ciertos eventos en un determinado tiempo.

Si bien el lanzamiento fue exitoso, comenzaron a aparecer reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas.

Uno de los principales problemas que impulsaron esta review es el endpoint de /exchanges, como se mostro es (TODO) este considerablemente mas lento que el resto de endpoints, lo que podria implicar el descontento de los usuarios.

las tasas de cambio no tienen gap entre el valor de compra y de venta.

Adicionalmente, el modelo de negocios es ofrecer una tasa de cambio sin gap, es decir en tiempo real. Una performance pobre en esta operacion implicaría que las tasas podrian que tener que ajustarse negativamente en comparacion a lo que se le ofrece al usuario en primera instancia.

2.1.3. Scalability (escalabilidad)

Habilidad del sistema para incorporar recursos adicionales de manera efectiva. Efectividad = mejora mensurable de algún QA + no requerir un esfuerzo inapropiado + no interrumpir el funcionamiento.

Para ganar usuarios, las tasas de cambio...

Se especifica que objetivo a corto plaso es incrementar el numero de usuarios, el cual, el sistema debe ser capaz de manejar incorporando los recursos necesarios.

2.1.4. Reliability (confiabilidad)

Grado en que la arquitectura es susceptible a fallas a nivel de sistema cuando se producen fallas parciales en componentes, conectores o datos.

TODO: TODO: justificar

2.1.5. Security (seguridad)

Capacidad del sistema de prevenir acciones maliciosas o accidentales fuera del uso esperado.

se valida que haya saldo en la cuenta del cliente, se espera que lo haga la UI

La posibilidad de que los usuarios puedan realizar acciones maliciosas implican perdidas monetarias para el resto de usuarios, para la propia organizacion, ademas de una perdida de reputacion y con esto perdida de usuarios e inversiones.

2.2. Otros QA menos relevantes

Ademas se mencionan otros QA, que aunque no resulten claves y por ende no son una prioridad, son desceables, especialmente conforme el proyecto crezca.

TODO: TODO: busca en el enunciado o en la documentacion como justificar estos QA

2.2.1. Efficiency

Performance por recurso consumido.

2.2.2. Testability (testeabilidad)

Grado en que un componente/sistema de software permite ser probado para exhibir sus fallas; usualmente relacionado al esfuerzo necesario para llevar adelante las tareas de testing.

2.2.3. Manageability

Define la facilidad de gestión del sistema por parte de los administradores del mismo.

2.2.4. Simplicity

Facilidad con la que la arquitectura de un sistema puede ser entendida, razonada y explicada.

2.3. Decisiones de diseño actuales y su impacto en los QA**2.3.1. Impacto en la Performance**

...

2.4. Critica**3. COMPORTAMIENTO DEL SERVICIO****3.1. Pruebas de carga****3.2. Metricas****3.2.1. Latencia**

...

4. MODIFICACIONES PROPUESTAS**4.1. Propuesta 1**

El sistema presenta una degradacion principalmente por el manejo indevido del log. Destacamos que la posibilidad de permitir consultar el log de manera completa es la principal causa de degradacion. Este escala con el numero de operacion, lo que provoca un aumento en el uso de recursos para mantenerlo, y un aumento en los tiempos de respuesta para consultarlo. Esto arrastra la degradacion a los otros endpoints, entre ellos exchange, el principalmente mencionado por los usuarios.

Destacamos que un endpoint que permite consultarlo todo por definicion no es escalable, recomendamos la implementacion de paginacion. Sinembargo, asumimos que esta es un requerimiento de negocio y que las mejoras propuestas no deben alterar la firma del sistema para con los usuarios. Se propone desacoplar el log para que la degradacion de este no afecte a los endpoints de negocio del sistema. Proponemos agregar un nodo dedicado, analogo a exchange-api para el manejo del log. Algunas optimizaciones de la implementacion que consideramos que vale la pena mencionar son:

- Remplazamos el uso del formato .json para el almacenamiento a .ndjson, de esta manera podemos agregar los registros al final sin reescribir todo el archivo.
- El archivo ya no se carga en memoria, se streamea el cliente directamente desde disco en batches. Destacamos que agregamos los caracteres [al inicio y] al final del stream para mantener la firma original con respecto al formato .json.
- Solo enviamos hasta el ultimo byte valido disponible al momento de recibida la consulta y verificamos que la ultima entrada sea valida (evita race condition con la escritura).
- Exchange api loguea directamente al nuevo nodo dedicado mediante una conexcion TCP dedicada.

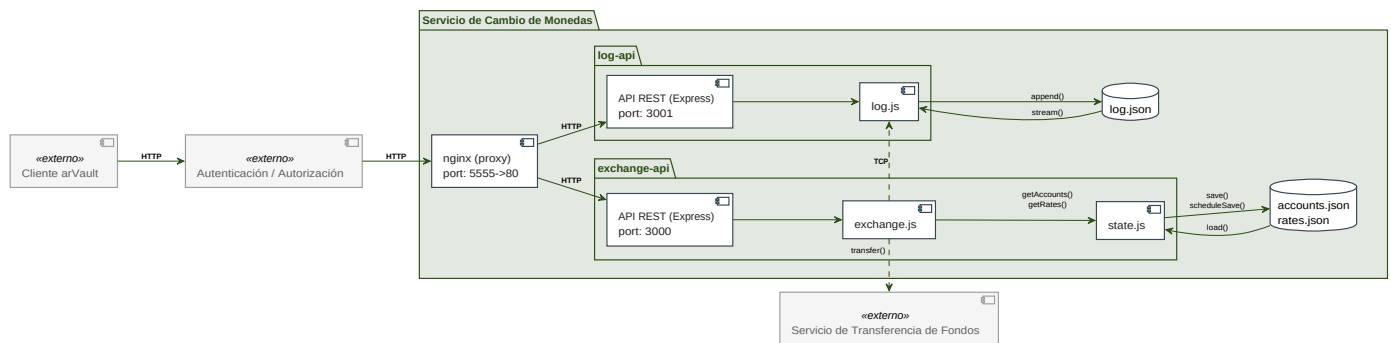


Figura 2. Diagrama Components & Connectors de la propuesta 1.

4.2. Impacto en los QA

4.2.1. Performance

TODO: evaluar el impacto en cada QA

4.3. Pruebas de carga

La prueba de carga se realizó usando una tasa constante de 40 usuarios durante 240s (4 minutos). Cada usuario realiza un solo request, es decir se mantiene un throughput constante de 40 requests por segundo. No se realizó warm-up ni ramp para no ensuciar las métricas de promedios. Tampoco se realizaron recargas de saldo intermedias, se utilizó una carga inicial suficiente para cubrir toda la prueba. Se pueden encontrar los resultados completos en el [Apendice](#).

4.4. Pruebas de estrés

La prueba de estrés se realizó incrementando progresivamente la carga desde 40 hasta 180 en saltos de 20, cada salto con una duración de 45s, el tiempo total de la prueba es de 450s (7,5 minutos). Se realizó un warm-up inicial de 5 usuarios durante 30s. Además, al final de la prueba se dejó una instancia de recovery de nuevamente 5 usuarios esta vez durante 60s. Tampoco se realizaron recargas de saldo intermedias y se mantuvo el mismo uso de endpoints para que los resultados sean comparables.

4.5. Métricas

4.5.1. Disponibilidad

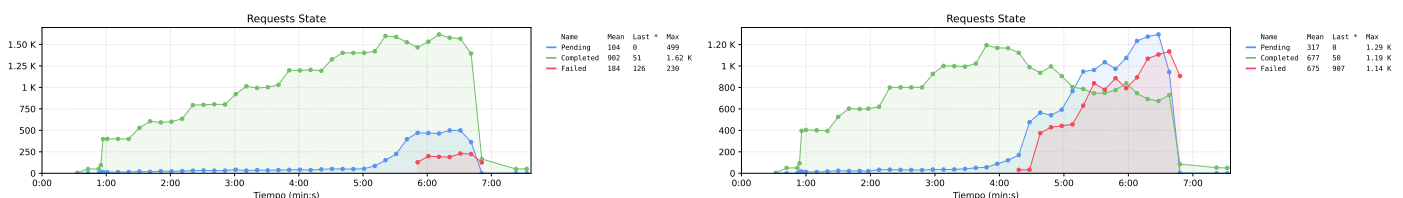


Figura 3. Comparativa de la disponibilidad ofrecida por la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de stress. La propuesta 1 obtuvo un incremento en el promedio de respuestas satisfactorias de 225 respuestas adicionales, lo que representa un aumento del 33,23 % frente al caso base.

4.5.2. Latencia

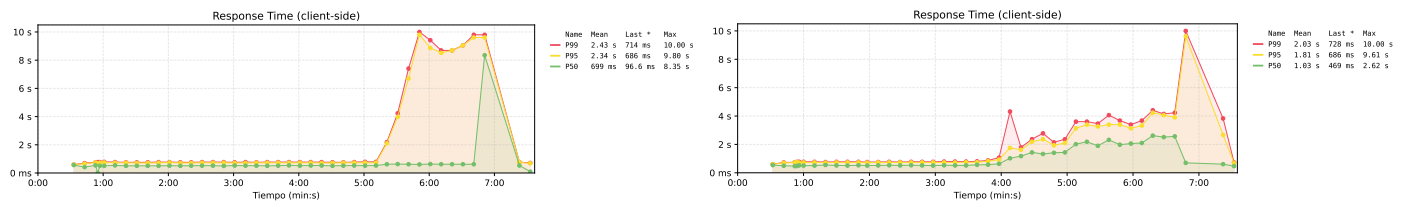


Figura 4. Comparativa de los resultados de latencia entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de stress. Se observa que la propuesta 1 mejoro el promedio de latencia para P50. Tambien se observa que empeoro en P95 y P99.

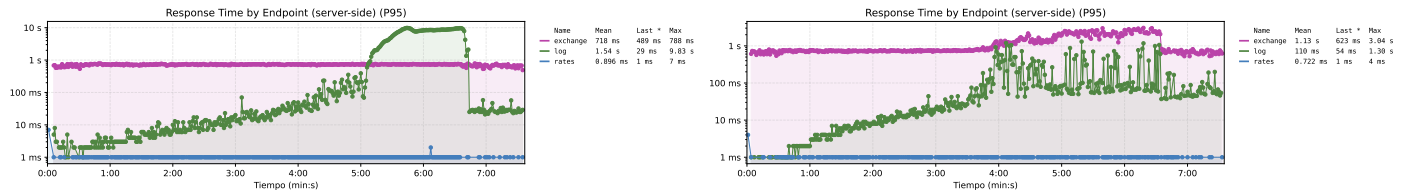


Figura 5. Comparativa de los resultados de latencia por endpoint entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de stress. Con diferencia el analisis mas interesante, la propuesta 1 permitio como se esperaba mejorar la latencia en el endpoint de exchange, se puede notar que este no se ve afectado por la degradacion del endpoint de logs. Como si ocurría en el caso base, donde al rededor del minuto 4, los tiempos de respuesta de exchange incrementaban, en este caso, los tiempos de respuesta se mantienen estables en 1 segundo, acotado por la dependencia del servicio externo. Por otro lado, los tiempos de respuesta de /log empeoraron, sinembargo es esperable, la mayor disponibilidad antes mencionada implica que se registraron mas transacciones, por lo que el tamaño del log necesariamente aumento. Esto explica el aumento de los tiempos de respuesta visto desde el lado del cliente antes mencionados, sinembargo, las metricas se ven ensuciadas al mezclar todos los endpoints.

4.5.3. Uso de recursos

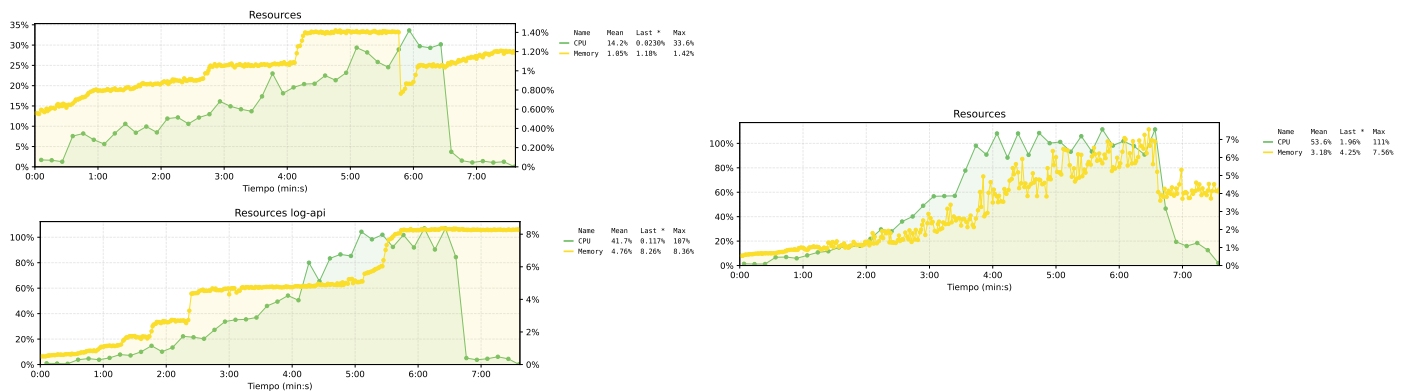


Figura 6. Comparativa de los resultados del uso de recursos entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de stress. Destacar que consideramos la comparacion con respecto a ambos nodos, el nodo de exchange y el de logs. Se puede observar que el nodo de logs mantiene la carga con respecto a la del sistema base, incluso la aumenta. Esto es esperable con el aumento de la disponibilidad, mas request completadas implican mas entradas en el log. Sinembargo, el nodo de exchange apenas llevo a un 14.2 % de uso de CPU y 1.42 % de uso de memoria.

4.5.4. Volumen operado por moneda

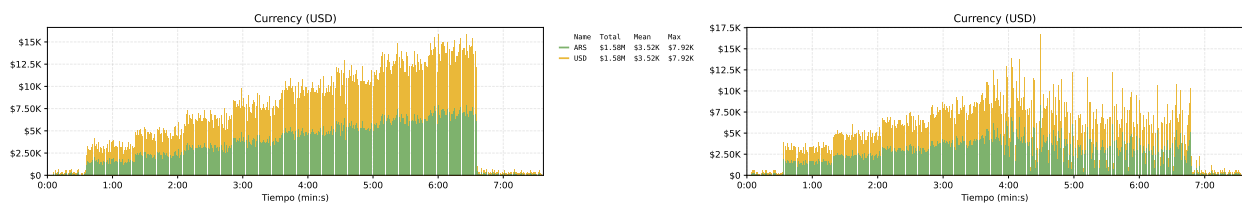


Figura 7. Comparativa de los resultados de volumen operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de stress. Se observa que la propuesta 1 obtuvo un mayor volumen total siendo este 1.58M frente al 1.18M del sistema base, es decir una mejora del 33.90 %.

4.5.5. Volumen neto operado

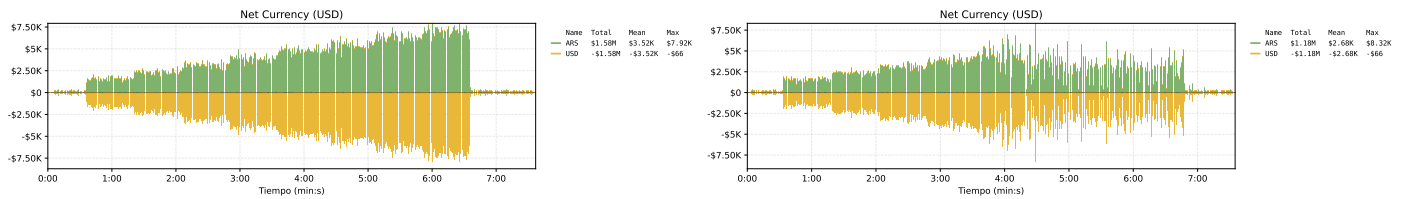


Figura 8. Comparativa de los resultados de volumen neto operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de stress. Análogamente al volumen operado, se observa que el volumen neto también es mayor en la propuesta 1 frente al sistema base.

4.6. Pruebas de resistencia

La prueba de resistencia se realizó usando una tasa constante de 40 usuarios durante 2700s (45 minutos). Se realizó un warm-up inicial de 5 usuarios durante 60s. Además, al final de la prueba se dejó una instancia de recovery de nuevamente 5 usuarios esta vez durante 120s (2 minutos). Tampoco se realizaron recargas de saldo intermedias y se mantuvo el mismo uso de endpoints para que los resultados sean comparables.

4.7. Métricas

4.7.1. Disponibilidad

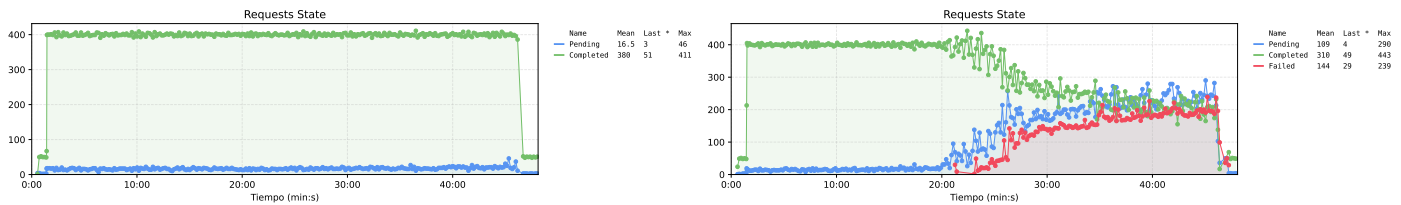


Figura 9.

TODO: completar

4.7.2. Latencia

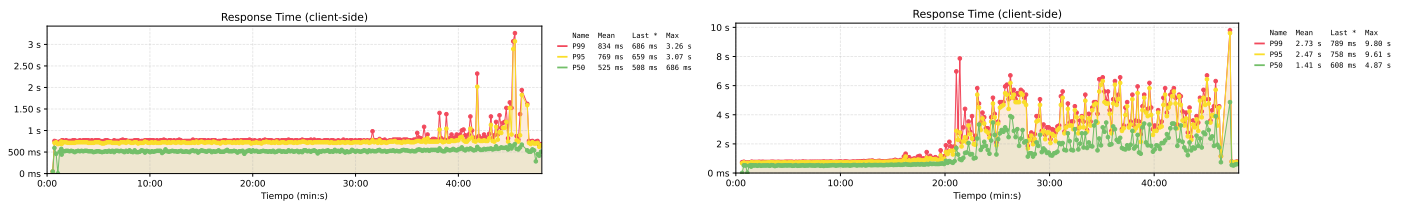


Figura 10.

TODO: completar

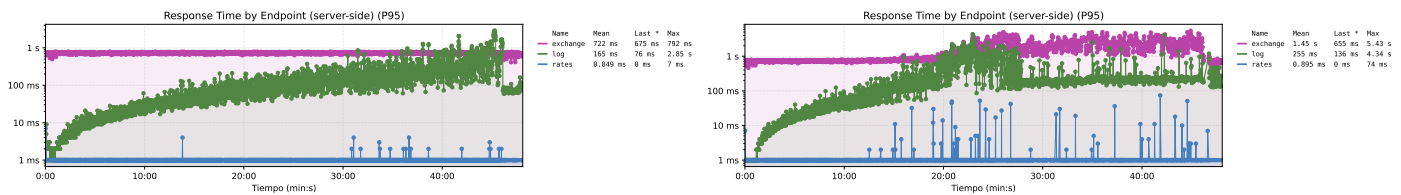


Figura 11.

TODO: completar

4.7.3. Uso de recursos

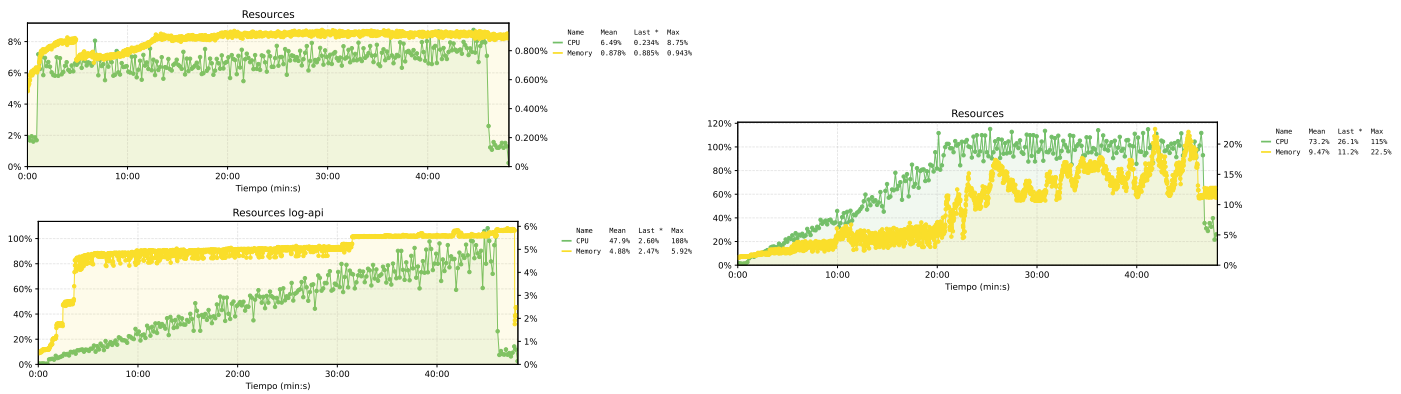


Figura 12.

TODO: completar

4.7.4. Volumen operado por moneda

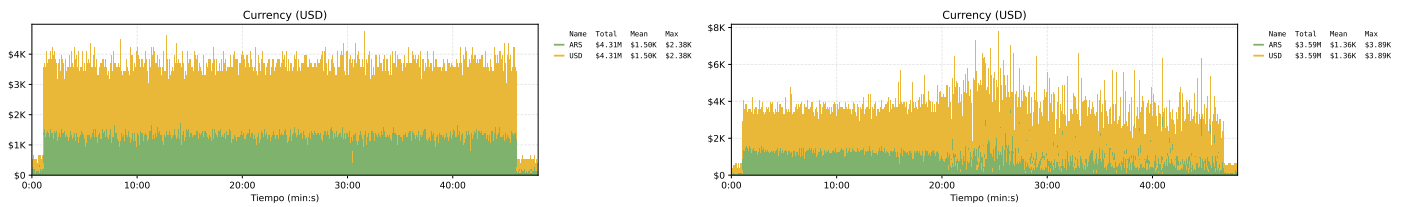


Figura 13.

TODO: completar

4.7.5. Volumen neto operado

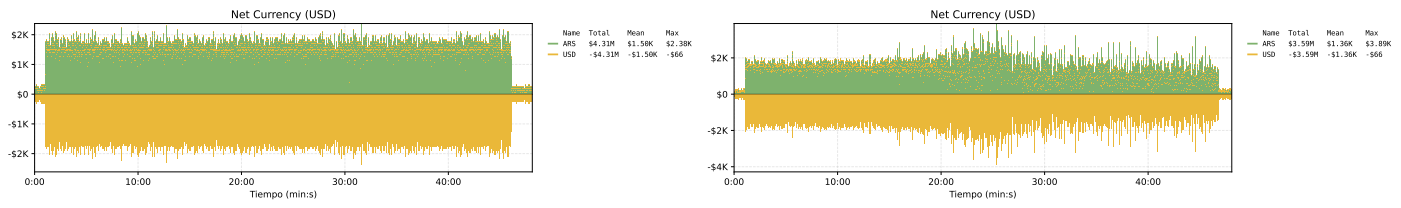


Figura 14.

TODO: completar

4.8. Resultados y conclusiones

La propuesta 1 demostro ser capaz de aislar la degradacion producida por la consulta de logs de las opeeraciones de negocio del sistema, permitiendo ofrecer una mejor disponibilidad a los usuarios comunes. Por otro lado la consulta de logs sigue degradandose con el tiempo. Sinembargo, este desacoplamiento propuesto permitiria replicar el nodo de consulta para dar soporte de lectura de los a mas usuarios, es decir, permitiendo escalabilidad horizontal.

4.9. Propuesta 2

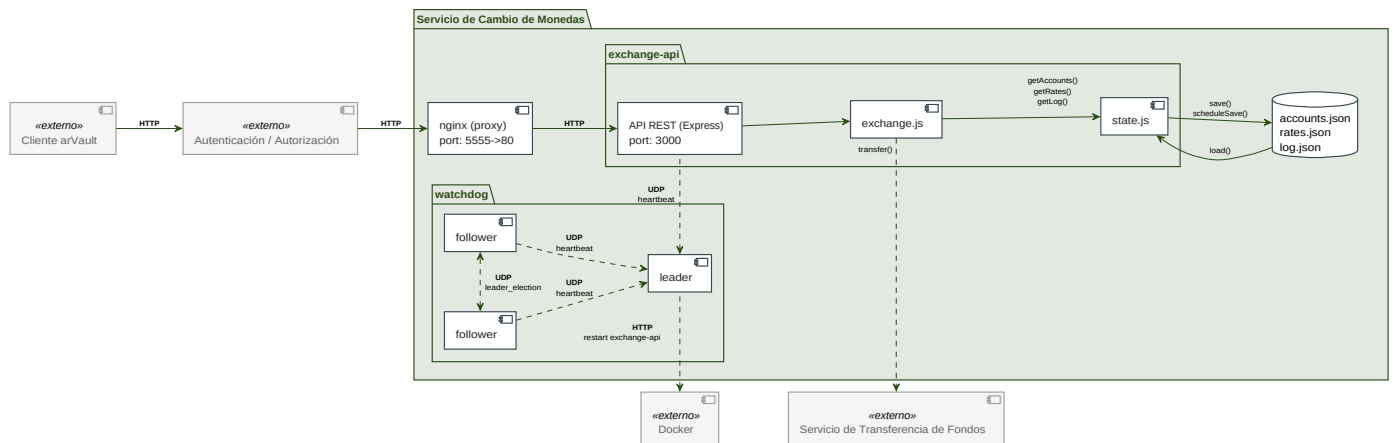


Figura 15

4.10. Impacto en los QA

4.10.1. Performance

4.11. Pruebas de carga

4.12. Pruebas de estrés

4.13. Métricas

4.13.1. Disponibilidad

4.13.2. Latencia

4.13.3. Uso de recursos

4.13.4. Volumen operado por moneda

4.13.5. Volumen neto operado

4.14. Pruebas de resistencia

4.15. Métricas

4.15.1. Disponibilidad

4.15.2. Latencia

4.15.3. Uso de recursos

4.15.4. Volumen operado por moneda

4.15.5. Volumen neto operado

4.16. Resultados y conclusiones

5. USO DE HERRAMIENTAS DE IA

...

A. APENDICE

A.1. Resultados completos del escenario plano sobre el sistema base

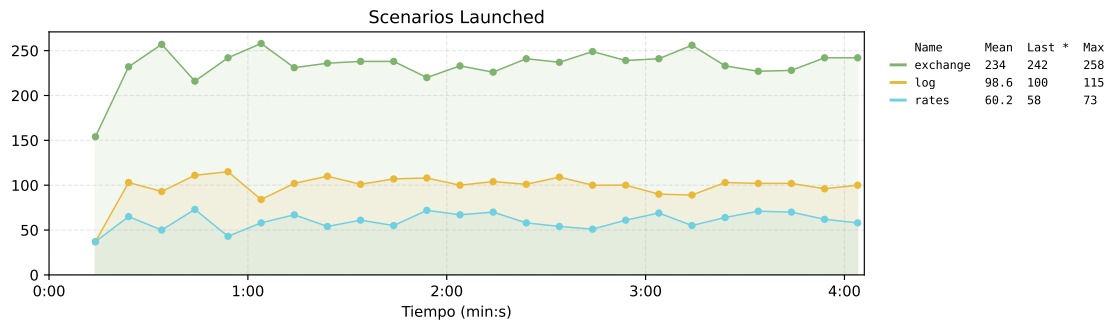


Figura 16. Escenarios del sistema base.

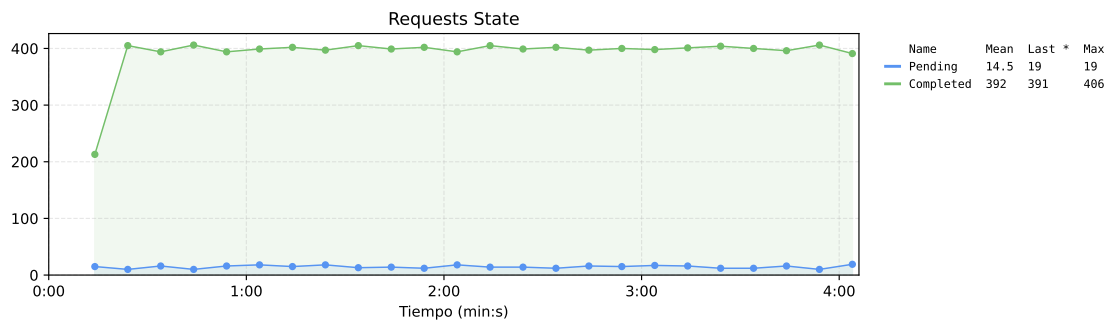


Figura 17. Estado de las requests del sistema base.

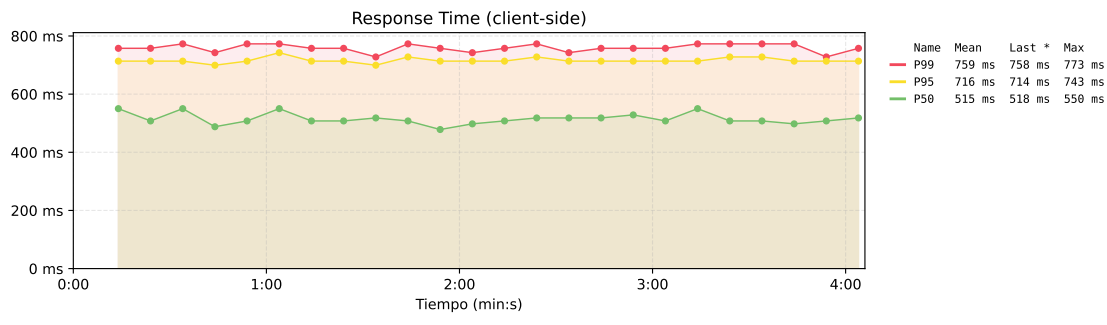


Figura 18. Tiempo de respuesta del sistema base.

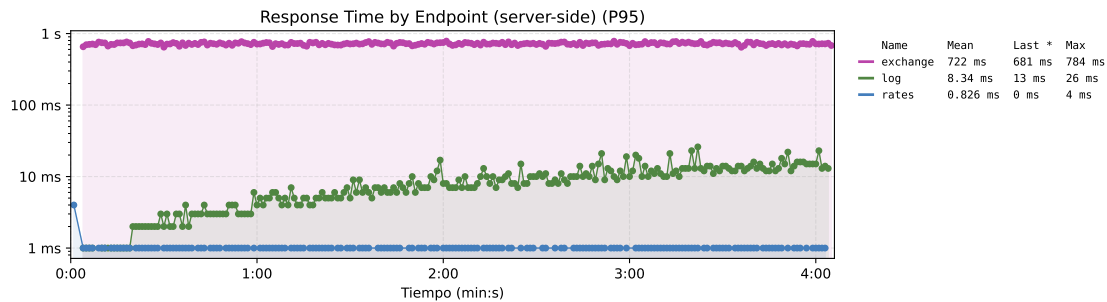


Figura 19. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logaritmica.

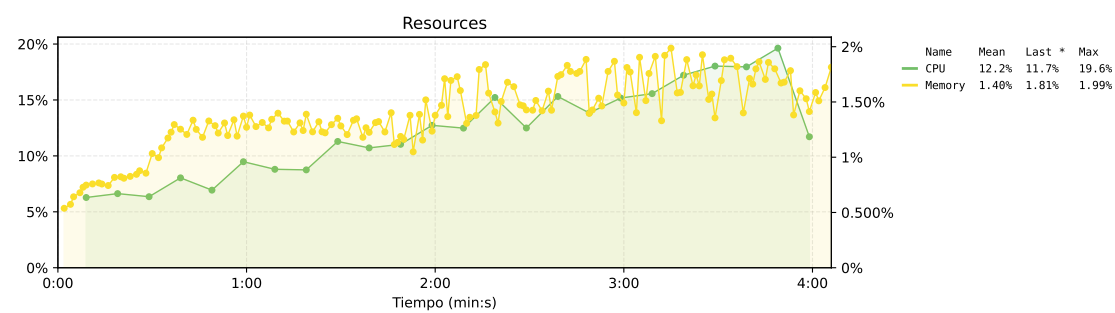


Figura 20. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

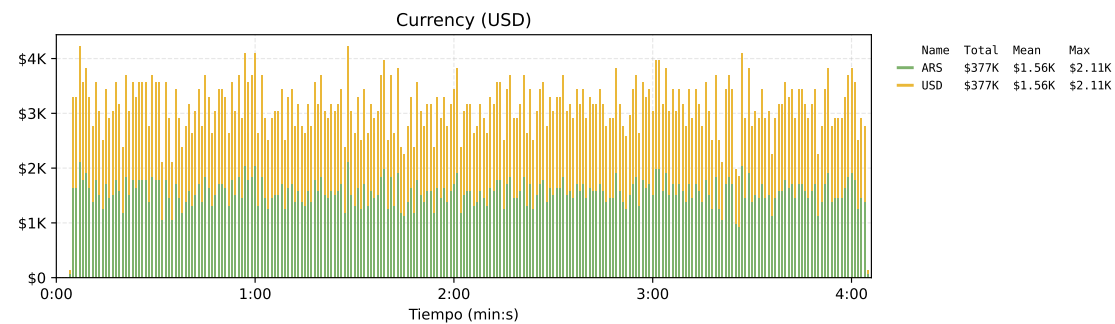


Figura 21. Volumen operado en cada moneda en el sistema base.

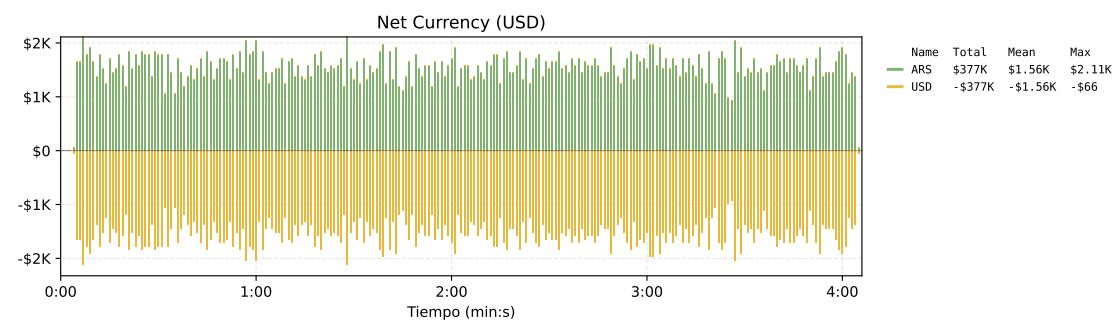


Figura 22. Neto operado en cada moneda en el sistema base.

A.2. Resultados completos del escenario stress sobre el sistema base

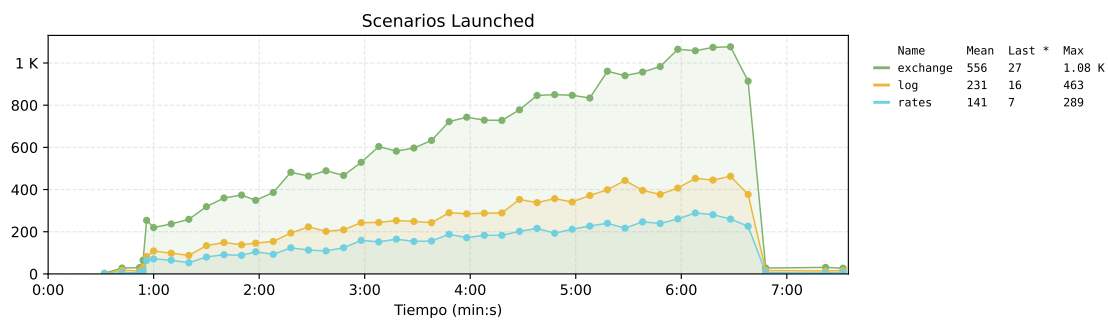


Figura 23. Escenarios del sistema base.

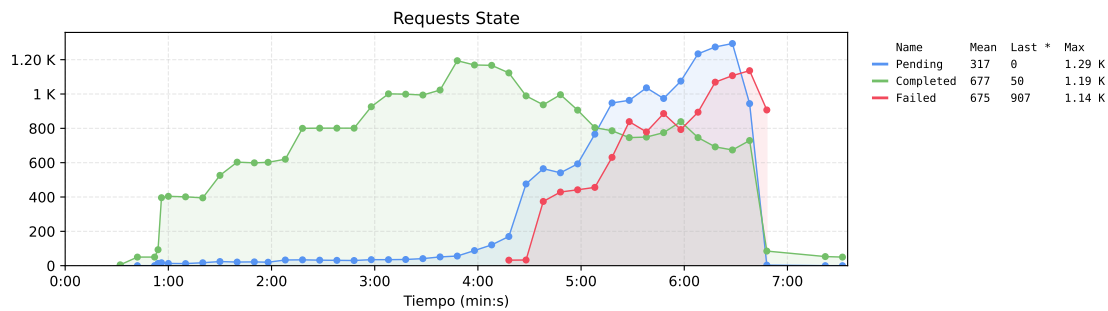


Figura 24. Estado de las requests del sistema base.

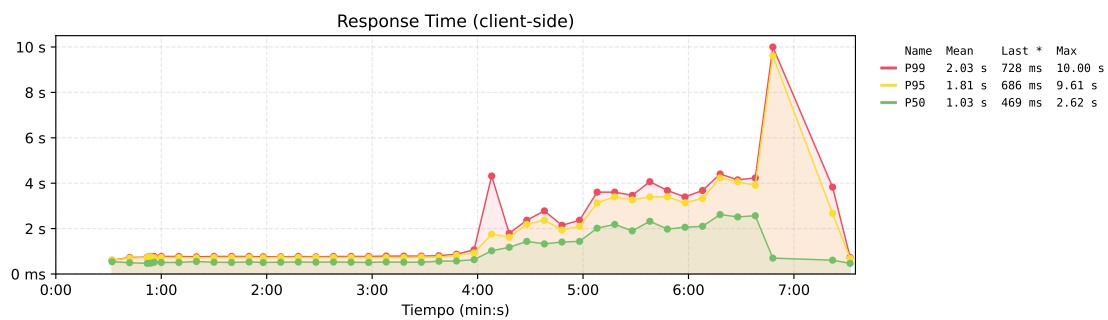


Figura 25. Tiempo de respuesta del sistema base.

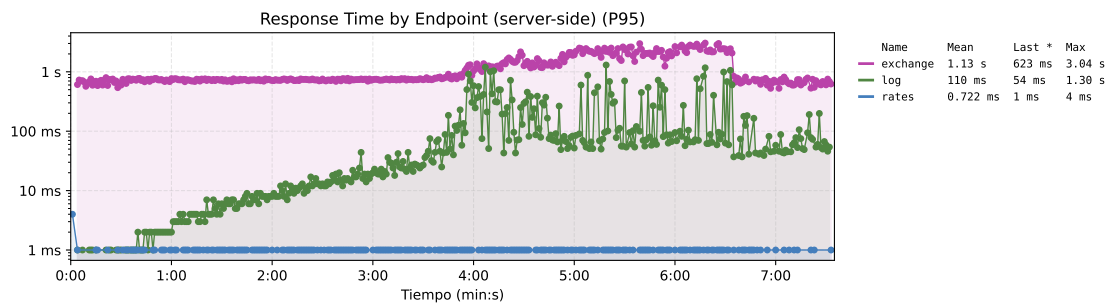


Figura 26. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logarítmica.

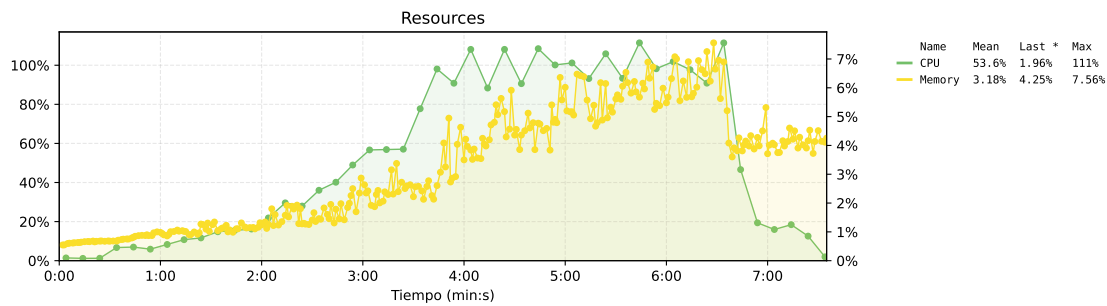


Figura 27. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

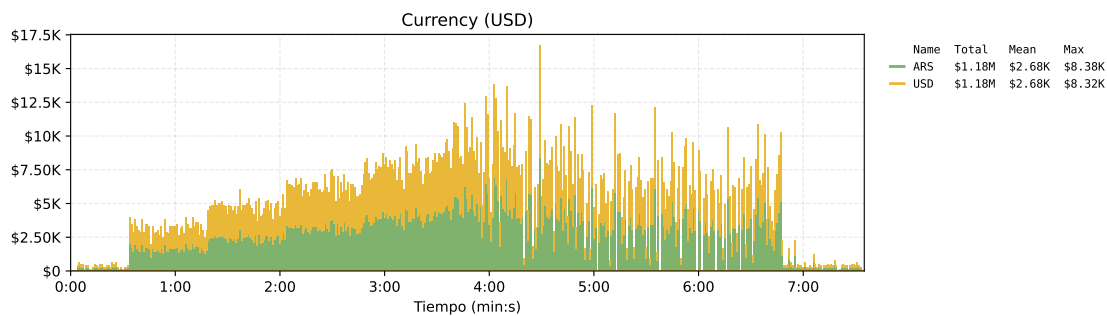


Figura 28. Volumen operado en cada moneda en el sistema base.

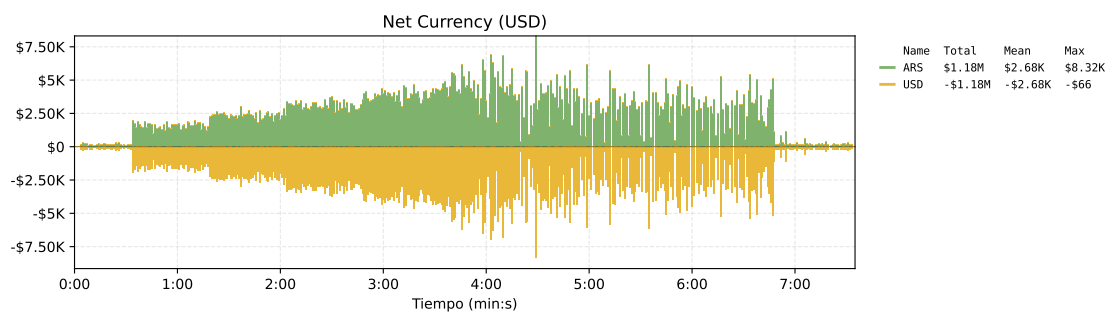


Figura 29. Neto operado en cada moneda en el sistema base.

A.3. Resultados completos del escenario soak sobre el sistema base

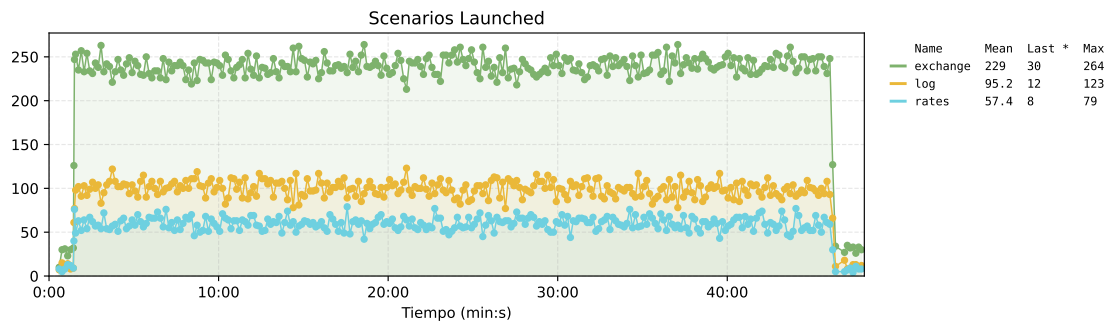


Figura 30. Escenarios del sistema base.

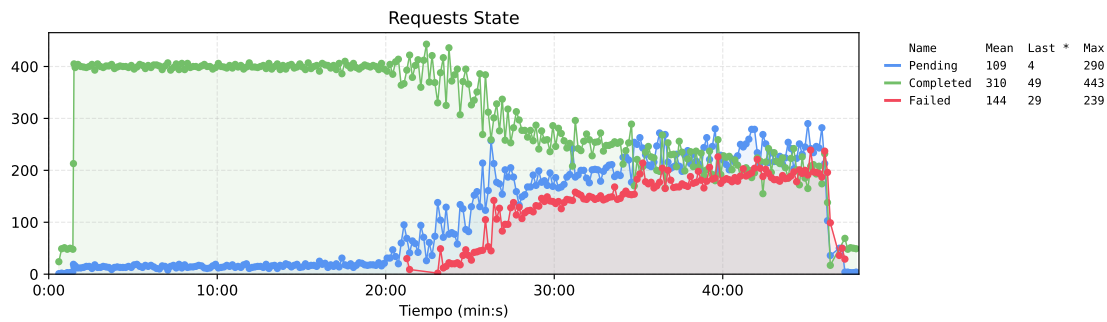


Figura 31. Estado de las requests del sistema base.

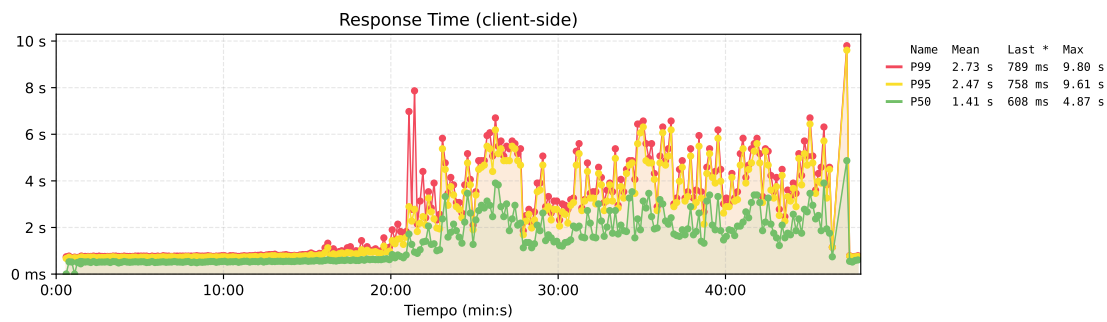


Figura 32. Tiempo de respuesta del sistema base.

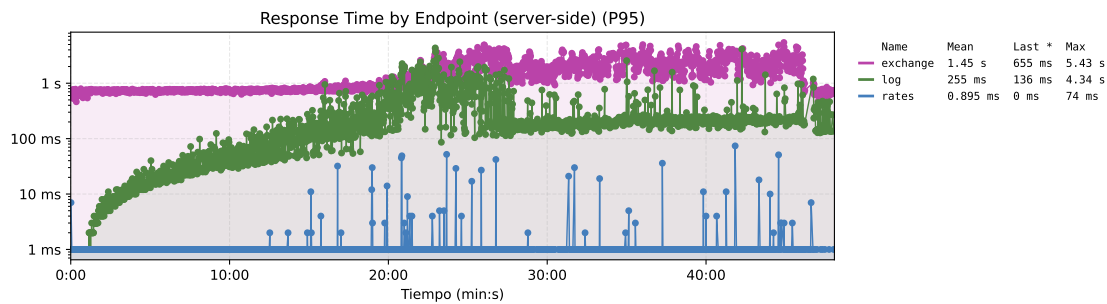


Figura 33. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logaritmica.

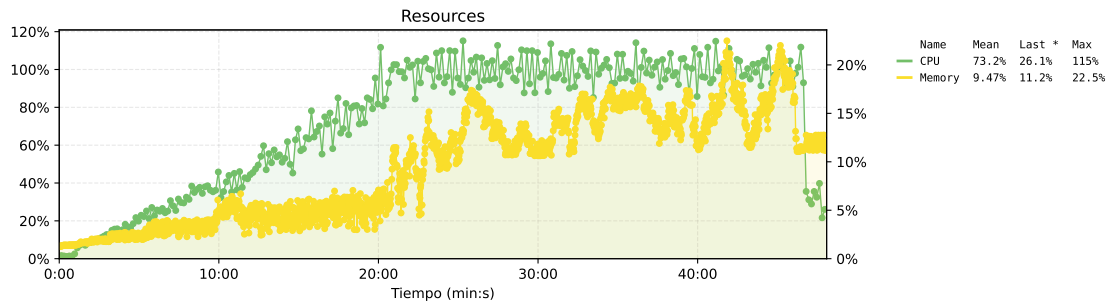


Figura 34. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

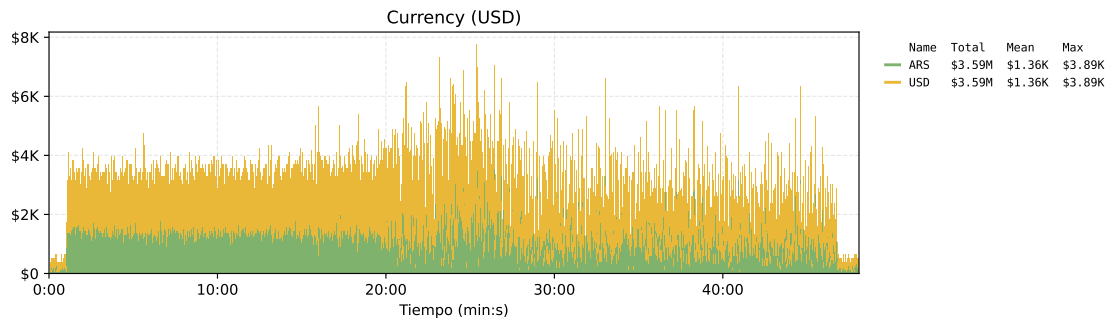


Figura 35. Volumen operado en cada moneda en el sistema base.

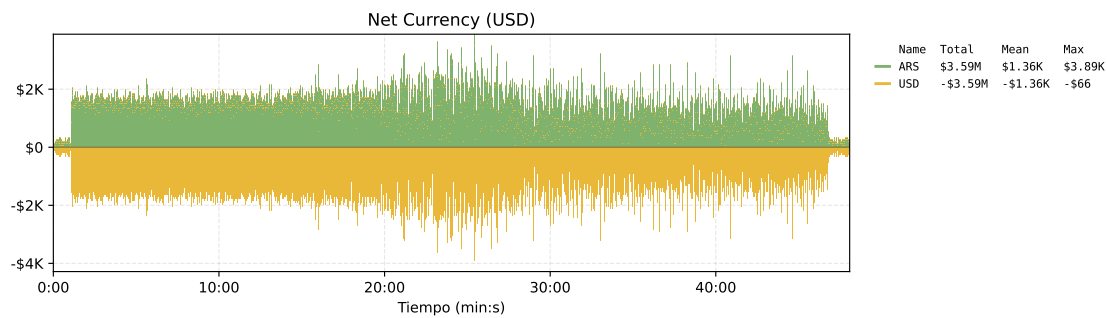


Figura 36. Neto operado en cada moneda en el sistema base.

A.4. Resultados completos del escenario plano sobre la propuesta 1

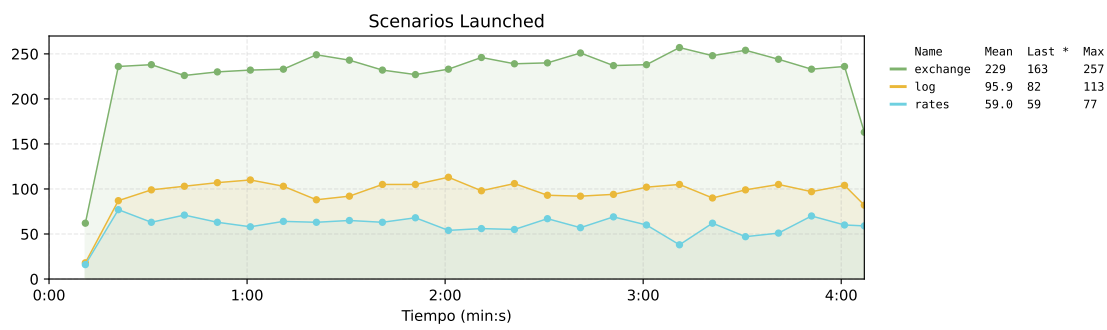


Figura 37. Escenarios de la propuesta 1.

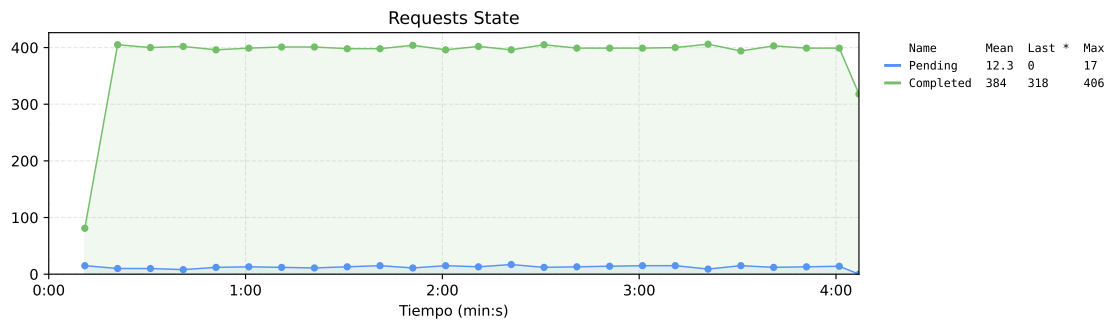


Figura 38. Estado de las requests de la propuesta 1.

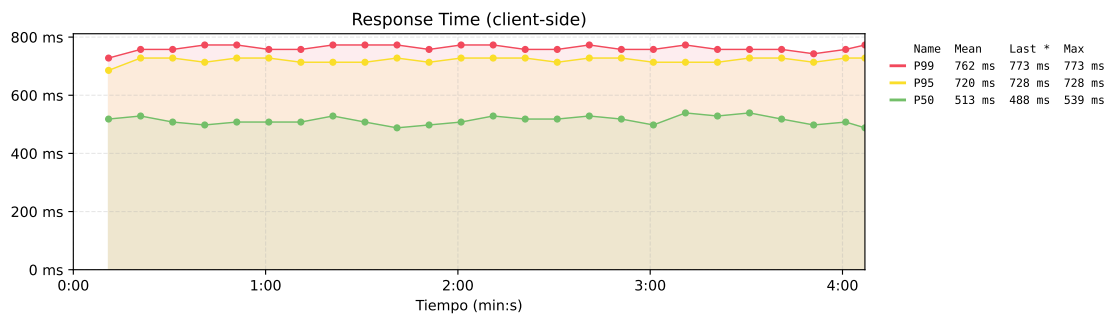


Figura 39. Tiempo de respuesta de la propuesta 1.

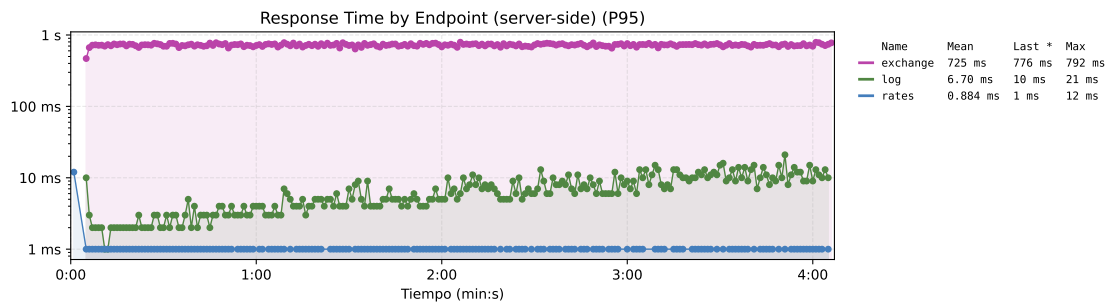


Figura 40. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logaritmica.

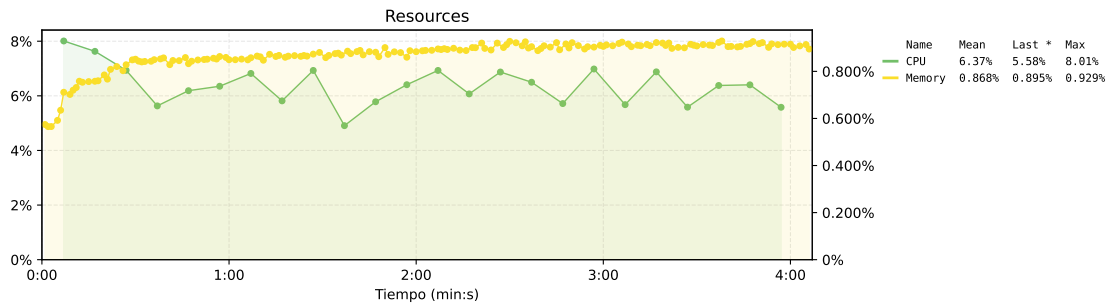


Figura 41. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

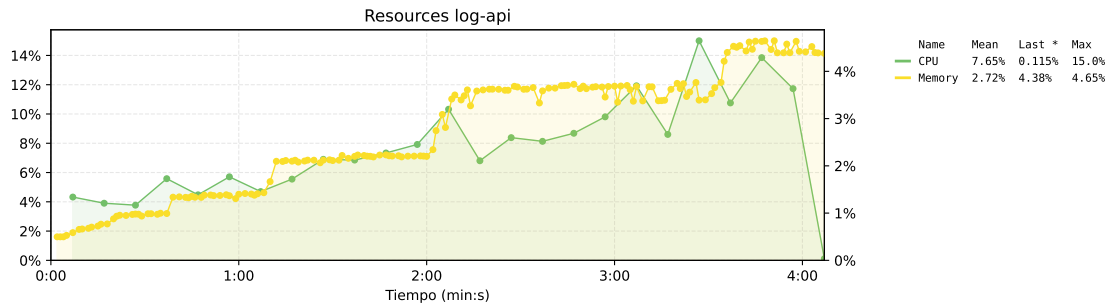


Figura 42. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

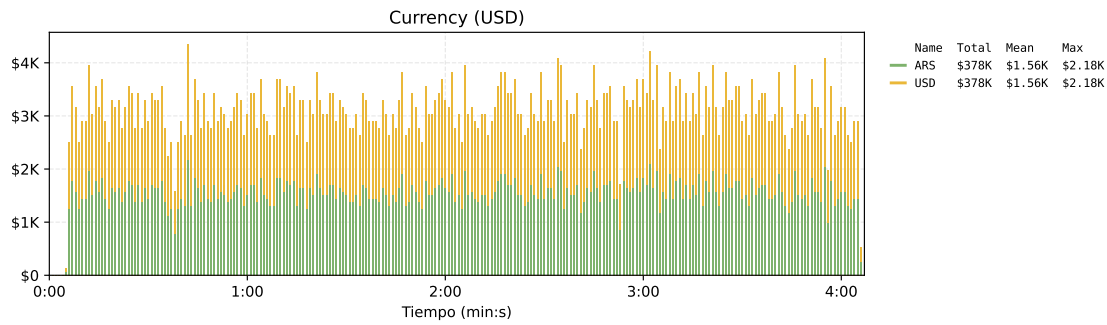


Figura 43. Volumen operado en cada moneda en la propuesta 1.

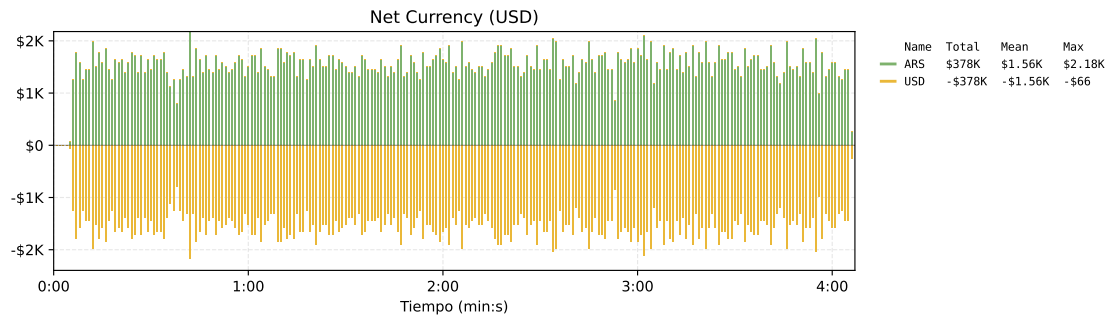


Figura 44. Neto operado en cada moneda en la propuesta 1.

A.5. Resultados completos del escenario stress sobre la propuesta 1

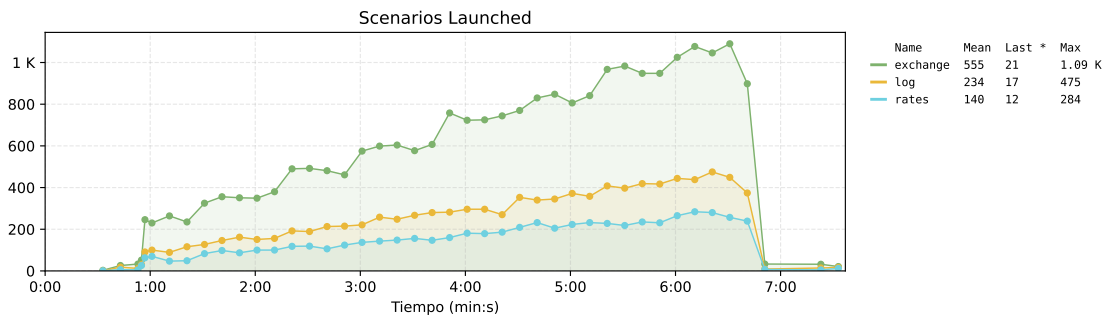


Figura 45. Escenarios de la propuesta 1.

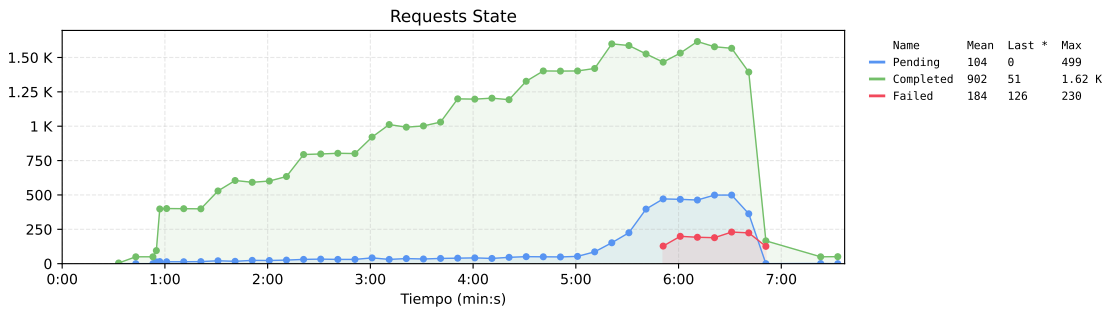


Figura 46. Estado de las requests de la propuesta 1.

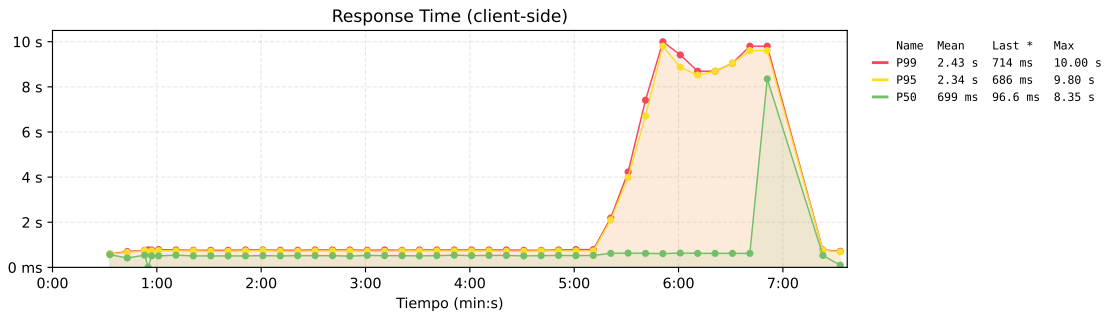


Figura 47. Tiempo de respuesta de la propuesta 1.

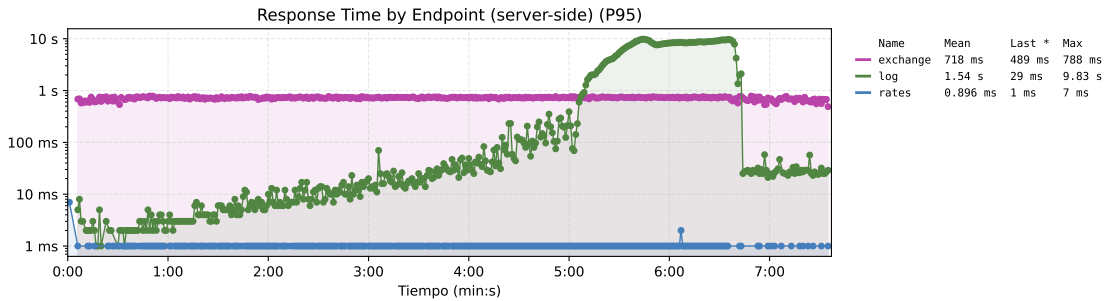


Figura 48. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logaritmica.

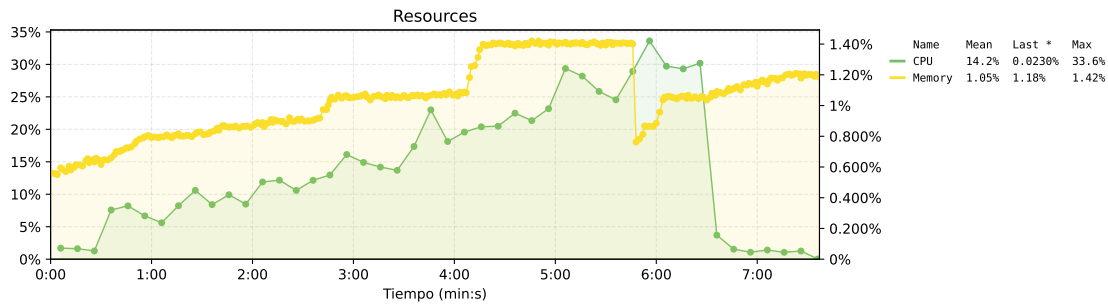


Figura 49. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

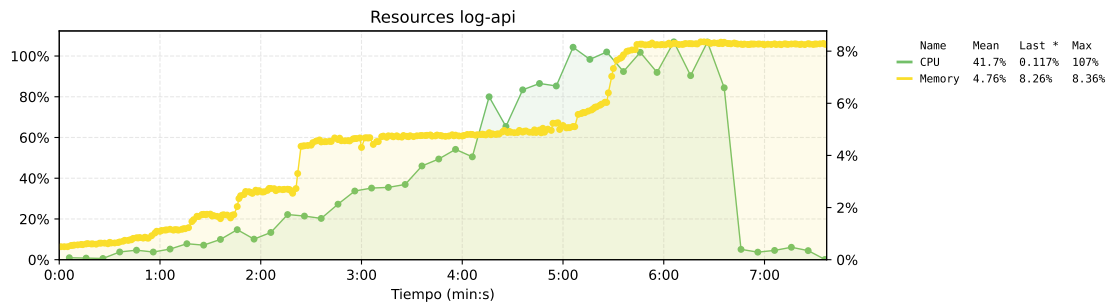


Figura 50. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

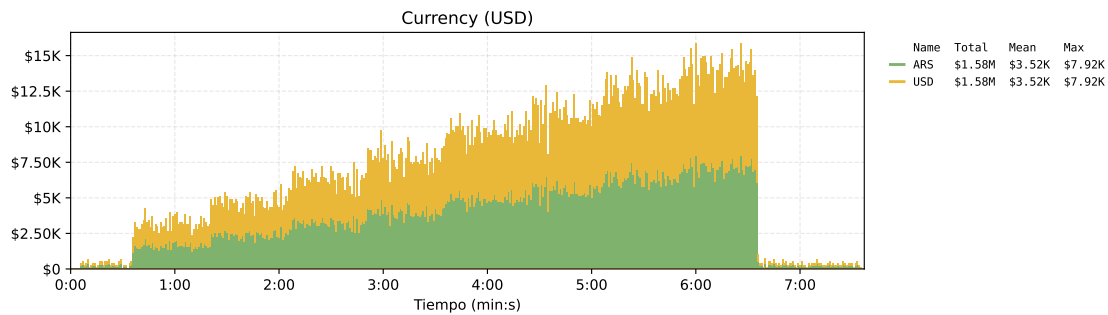


Figura 51. Volumen operado en cada moneda en la propuesta 1.

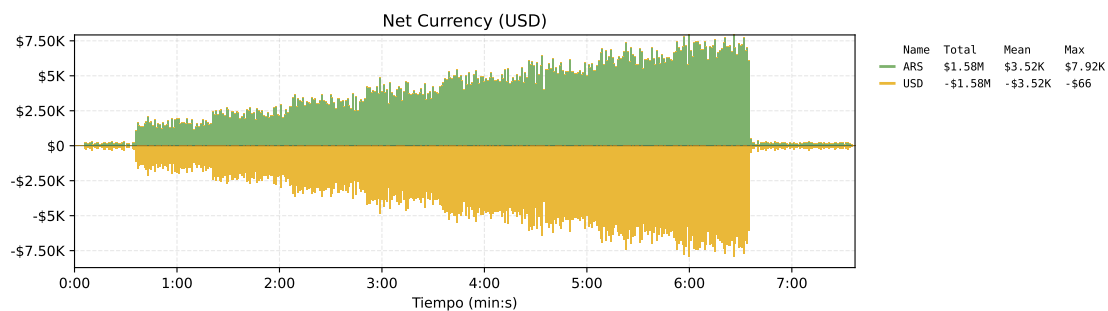


Figura 52. Neto operado en cada moneda en la propuesta 1.

A.6. Resultados completos del escenario soak sobre la propuesta 1

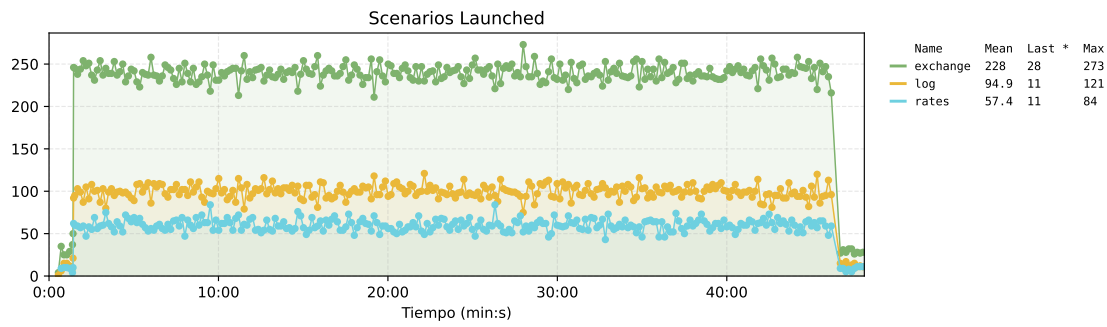


Figura 53. Escenarios de la propuesta 1.

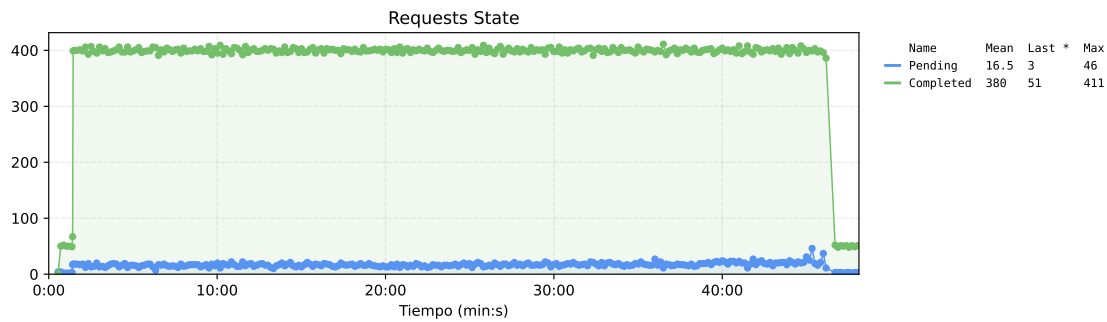


Figura 54. Estado de las requests de la propuesta 1.

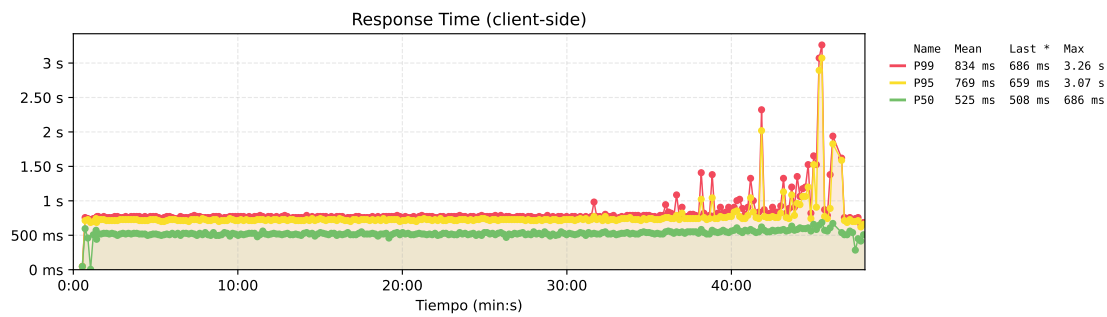


Figura 55. Tiempo de respuesta de la propuesta 1.

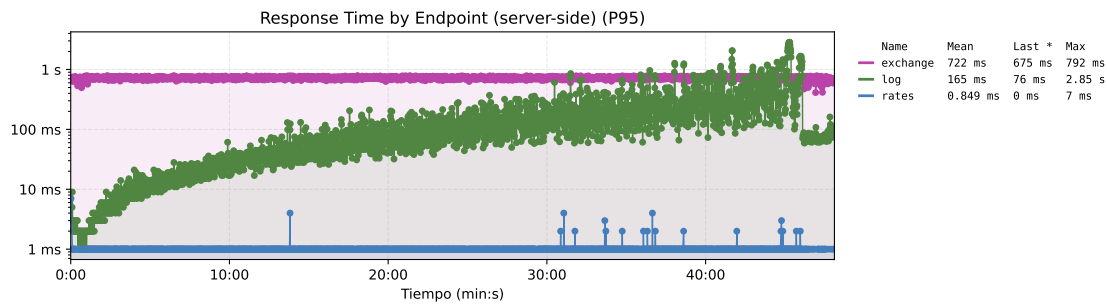


Figura 56. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logarítmica.

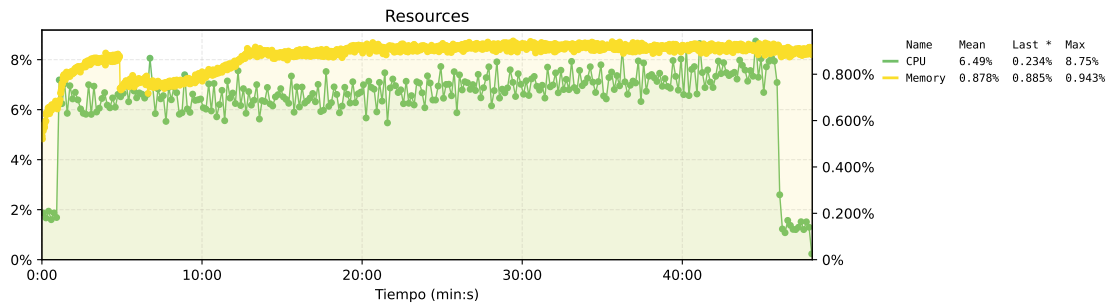


Figura 57. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

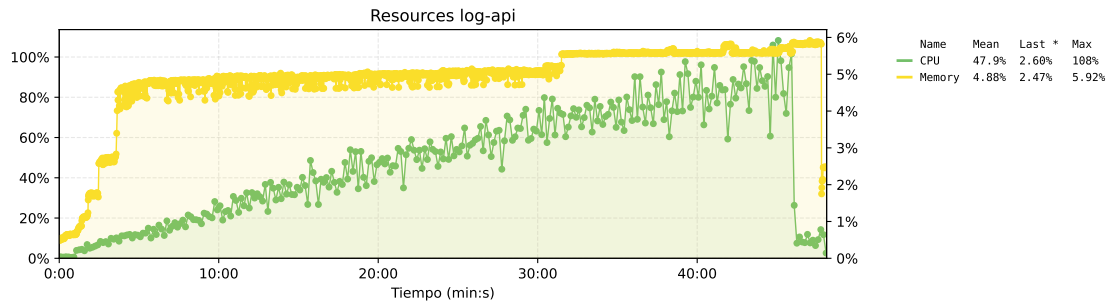


Figura 58. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

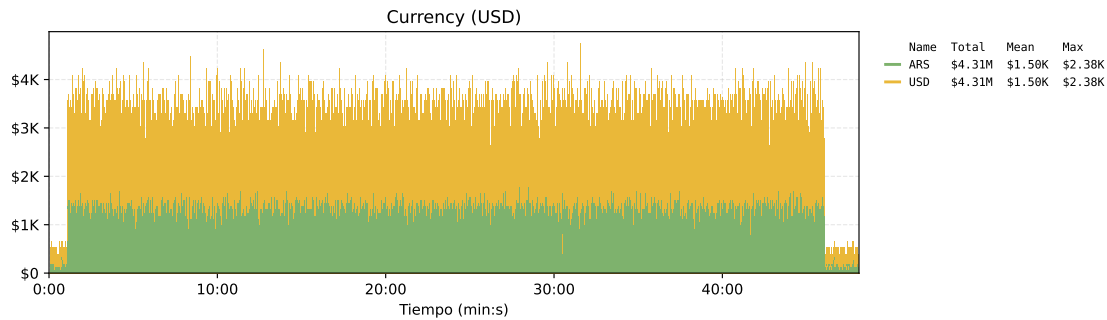


Figura 59. Volumen operado en cada moneda en la propuesta 1.

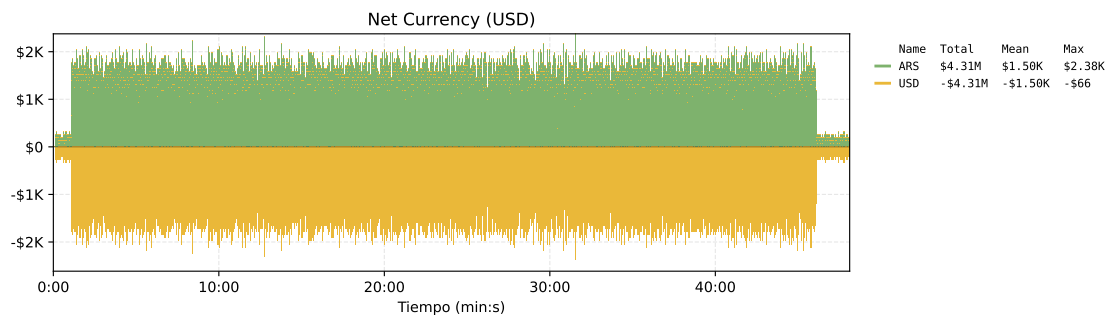


Figura 60. Neto operado en cada moneda en la propuesta 1.

A.7. Resultados completos del escenario plano sobre la propuesta 2

A.8. Resultados completos del escenario stress sobre la propuesta 2

A.9. Resultados completos del escenario soak sobre la propuesta 2